KePIN: Appendix

1 Composite Physics-Aware Loss

KePIN is trained by minimising a composite loss whose components enforce multiple complementary objectives simultaneously. The loss operates in two modes selected automatically: a *degradation mode* with seven components for predictive maintenance, and a *forecasting mode* with four components for all other domains.

Notation and setting. Throughout this section, we assume a mini-batch (or dataset aggregate) of N sequences, each of length T. For the i-th sequence, the encoder produces a latent trajectory $\{\mathbf{z}_t^{(i)}\}_{t=1}^{T}$ with latent dimension d_z (i.e., $\mathbf{z}_t^{(i)} \in \mathbb{R}^{d_z}$). The Koopman operator is a matrix $\mathbf{K} \in \mathbb{R}^{d_z \times d_z}$ intended to advance latent states by one step, $\mathbf{z}_{t+1} \approx \mathbf{K}\mathbf{z}_t$. The supervised target is y_i with corresponding prediction $\hat{y}_i$. For degradation-specific terms, we use two time indices $t_{\text{early}} < t_{\text{late}}$ within a sequence and write $\hat{y}_{\text{early}}^{(i)} := \hat{y}^{(i)}(t_{\text{early}})$ and $\hat{y}_{\text{late}}^{(i)} := \hat{y}^{(i)}(t_{\text{late}})$ (analogously for y). The time gap is $\Delta t := t_{\text{late}} - t_{\text{early}}$ (in the same discrete time units used by the sequence).

What each loss "means". Each component is designed to encode a simple physical or operational principle: (i) prediction accuracy ($\mathcal{L}_{\text{pred}}$); (ii) existence of a consistent linear latent dynamics model ($\mathcal{L}_{\text{koop}}$); (iii) dynamical stability of the linear evolution ($\mathcal{L}_{\text{spec}}$); (iv) long-horizon faithfulness of the learned dynamics ($\mathcal{L}_{\text{multi}}$); and for degradation tasks: (v) irreversibility ($\mathcal{L}_{\text{mono}}$); (vi) risk-aware asymmetry ($\mathcal{L}_{\text{asym}}$); (vii) correct degradation rate ($\mathcal{L}_{\text{slope}}$). The remainder of this appendix expands these principles into precise objectives and provides short proofs for the stability claims that motivate them.

1.1 Universal Loss Components

1. Prediction Loss $\mathcal{L}_{pred}$. The primary component measures prediction accuracy using the Huber loss, which is less sensitive to outliers than mean squared error:

$$\mathcal{L}_{\text{pred}} = \frac{1}{N}\sum_{i=1}^{N} H_\delta(\hat{y}_i - y_i), \quad H_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta(|r| - \frac{\delta}{2}) & |r| > \delta \end{cases} \tag{1}$$

with $\delta = 1.0$. For small residuals the loss is quadratic; for large residuals it grows linearly, preventing outliers from dominating training. *Interpretation.* Writing the residual as $r_i = \hat{y}_i - y_i$, the Huber loss behaves like mean squared error when $|r_i|$ is small, but transitions to an absolute-error-like regime when $|r_i|$ is large. This retains high precision near the optimum while reducing the influence

of rare but extreme errors. *Role in the composite objective.* All other loss terms constrain the latent dynamics; $\mathcal{L}_{\text{pred}}$ ensures the overall model remains anchored to the supervised task metric (RUL in degradation mode, or forecasting targets otherwise).

Practical note (scale). Because $\mathcal{L}_{\text{pred}}$ is a scalar loss in the target's physical units (cycles for RUL, degrees for temperature, etc.), its magnitude can differ substantially across datasets. This is one reason we use automatic uncertainty-based balancing (Section 1.3) rather than fixed manual coefficients.

2. Koopman Consistency Loss $\mathcal{L}_{koop}$. This component enforces the one-step linear transition $\mathbf{K}\mathbf{z}_t \approx \mathbf{z}_{t+1}$ by penalising the discrepancy between the Koopman-predicted and encoder-produced next latent states:

$$\mathcal{L}_{\text{koop}} = \frac{1}{N(T-1)} \sum_{i=1}^{N} \sum_{t=1}^{T-1} \|\mathbf{K}\mathbf{z}_t^{(i)} - \mathbf{z}_{t+1}^{(i)}\|_2^2 \tag{2}$$

This acts as a *linear dynamics prior*: it ensures the encoder and operator together learn a genuine Koopman decomposition, where consecutive latent states are related by a fixed linear map. This is a domain-agnostic physical constraint—linearity of the lifted dynamics is guaranteed to hold for any physical system in a sufficiently rich observable space [2]. *Interpretation.* For each sequence i and timestep t, the term inside the norm compares (a) the next latent state predicted purely by linear evolution, $\mathbf{K}\mathbf{z}_t^{(i)}$, and (b) the next latent state produced by encoding the next observation, $\mathbf{z}_{t+1}^{(i)}$. Minimising $\mathcal{L}_{\text{koop}}$ couples the encoder and Koopman operator so that the latent trajectory is *self-consistent* with the linear dynamics model. *Why one-step consistency matters.* If $\mathbf{K}\mathbf{z}_t \approx \mathbf{z}_{t+1}$ holds across many t, then repeated application implies $\mathbf{K}^h\mathbf{z}_t \approx \mathbf{z}_{t+h}$ for larger horizons h; this is the central mechanism by which a single learned operator can explain temporal evolution.

Connection to system identification. Eq. (2) can be viewed as a least-squares identification objective in latent space: it pushes the latent transition pairs $(\mathbf{z}_t, \mathbf{z}_{t+1})$ to be explained by a *single* linear operator. When this is satisfied, the learned representation behaves like a lifted linear state-space model, which is the operational meaning of Koopman linearisation.

3. Spectral Stability Loss $\mathcal{L}_{spec}$. While the SVD parameterisation bounds singular values, eigenvalue magnitudes of a non-normal matrix can differ from singular values. This soft complement directly penalises eigenvalues outside the unit circle:

$$\mathcal{L}_{\text{spec}} = \frac{1}{d_z} \sum_{j=1}^{d_z} \max(0, |\lambda_j| - s_{\text{spec}})^2, \qquad s_{\text{spec}} \leq 1 \tag{3}$$

Setting $s_{\text{spec}} = 1$ recovers the common "unit-circle" penalty; using a margin $s_{\text{spec}} < 1$ encourages eigenvalues to lie *strictly* inside the unit circle, improving

numerical robustness. Eigenvalues inside the margin contribute zero; violations receive quadratically increasing penalties. This enforces *dissipative stability* as a physics prior: physical systems with bounded energy cannot exhibit exponentially growing dynamics. *Interpretation.* For discrete-time linear dynamics $\mathbf{z}_{t+1} = \mathbf{K}\mathbf{z}_t$, eigenvalues with magnitude $|\lambda| > 1$ correspond to modes that grow exponentially under repeated application of $\mathbf{K}$. Penalising only the amount by which $|\lambda_j|$ exceeds one makes the constraint "inactive" when stability is satisfied, while providing a smooth training signal when it is violated. *Why eigenvalues (not only singular values).* Bounding singular values controls how much $\mathbf{K}$ can expand vectors in the worst case, but for non-normal matrices the eigenvalue magnitudes can still indicate unstable modal behavior that is not directly captured by singular values alone. The spectral penalty is therefore included as a targeted stability regulariser.

Why we use both singular-value and eigenvalue control. For a general (possibly non-normal) matrix, eigenvalues govern *asymptotic* modal growth/decay, but the singular values govern worst-case amplification of arbitrary directions. A non-normal matrix can have all eigenvalues inside the unit circle yet still exhibit transient growth (large intermediate amplification) before eventually decaying. Bounding singular values (via the SVD parameterisation) prevents such worst-case amplification, while the eigenvalue margin term acts as an additional safeguard against unstable modal structure.

4. Multi-Step Rollout Loss $\mathcal{L}_{multi}$. One-step accuracy does not guarantee multi-step accuracy. This component directly enforces long-horizon dynamical fidelity over horizons $\mathcal{H} = \{2, 3, 5\}$:

$$\mathcal{L}_{\text{multi}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \frac{1}{N(T-h)} \sum_{i,t} \|\mathbf{K}^h \mathbf{z}_t^{(i)} - \mathbf{z}_{t+h}^{(i)}\|_2^2 \tag{4}$$

Interpretation. This loss explicitly checks whether the same operator $\mathbf{K}$ can be rolled out for multiple steps without drifting away from the encoder-produced latent trajectory. Concretely, for each horizon h and timestep $t \leq T-h$, it compares the h-step prediction $\mathbf{K}^h\mathbf{z}_t$ against the "ground truth" latent state $\mathbf{z}_{t+h}$. *Why multi-step terms are needed.* Even when one-step errors are small, repeatedly applying a slightly biased operator can accumulate error across time. By including multiple horizons $h \in \{2, 3, 5\}$, training directly discourages compounding drift and encourages dynamics that remain faithful over longer rollouts.

Relation to stability. When $\|\mathbf{K}\|_2 < 1$, the linear rollout is contractive (formal proof in Section 1.4), which makes multi-step prediction numerically well-conditioned: errors do not blow up exponentially with horizon. The multi-step loss then focuses learning on *matching the true latent evolution* rather than fighting instability.

1.2 Degradation-Specific Components

5. Monotonicity Loss $\mathcal{L}_{mono}$. Penalises predicted RUL increases, encoding the physical irreversibility of degradation:

$$\mathcal{L}_{\text{mono}} = \frac{1}{N}\sum_{i=1}^{N} \max\Big(0,\, \hat{y}^{(i)}_{\text{late}} - \hat{y}^{(i)}_{\text{early}}\Big)^2 \tag{5}$$

Interpretation. For remaining useful life (RUL), the value should not increase as time advances under normal operation. The hinge-like form $\max(0, \cdot)$ means only violations (increases) are penalised; if $\hat{y}^{(i)}_{\text{late}} \leq \hat{y}^{(i)}_{\text{early}}$, the contribution is exactly zero. *Effect.* This term does not force a particular decay rate; it enforces only the directionality (non-increasing behavior) that reflects irreversibility of wear.

Implementation detail. In practice, t_{early} and t_{late} can be sampled within each training window, or chosen deterministically (e.g., first/last indices). Sampling provides more pairwise constraints per sequence and tends to improve enforcement of monotonicity without requiring longer sequences.

6. Asymmetric Penalty Loss $\mathcal{L}_{asym}$. Encodes the operational asymmetry that over-estimated RUL carries higher maintenance risk than under-estimated RUL:

$$\mathcal{L}_{\text{asym}} = \frac{1}{N}\sum_{i=1}^{N} \ell_{\text{asym}}(\hat{y}_i - y_i), \quad \ell_{\text{asym}}(r) = \begin{cases} 1.0\, r^2 & r \leq 0 \\ 2.5\, r^2 & r > 0 \end{cases} \tag{6}$$

Interpretation. Let $r = \hat{y} - y$ be the prediction residual. For RUL, $r > 0$ corresponds to over-estimation (predicting too much remaining life) and $r \leq 0$ to under-estimation. The larger coefficient for $r > 0$ (2.5) makes over-estimation more costly during training, aligning the model with risk-aware maintenance priorities.

Why this is "physics-aware". This term is not derived from a governing equation; instead it reflects a real operational constraint in maintenance decision-making: late maintenance can be catastrophic, while early maintenance is usually only costly. Encoding this asymmetry improves trustworthiness of RUL outputs in deployment.

7. Slope Matching Loss $\mathcal{L}_{slope}$. Enforces that the predicted degradation rate matches the true rate, preventing compounding bias over long prediction horizons:

$$\mathcal{L}_{\text{slope}} = \frac{1}{N}\sum_{i=1}^{N} \left(\frac{\hat{y}^{(i)}_{\text{late}} - \hat{y}^{(i)}_{\text{early}}}{\Delta t} - \frac{y^{(i)}_{\text{late}} - y^{(i)}_{\text{early}}}{\Delta t} \right)^2 \tag{7}$$

Interpretation. This term compares finite-difference slopes over the same interval Δt. Matching slopes encourages the model to capture the *rate* of degradation in addition to absolute RUL values, which helps reduce systematic drift across time. *Why it complements monotonicity.* $\mathcal{L}_{\text{mono}}$ enforces that RUL does not increase, but does not prevent the model from decreasing too slowly or too quickly. Slope matching penalises such rate mismatches directly.

Practical note (finite differences). Slope matching uses a finite-difference approximation. It is intentionally simple and robust: it does not require differentiating through time, and it directly targets the observable quantity of interest (RUL rate) rather than an unobserved internal state.

1.3 Automatic Loss Balancing

The composite loss combines either four or seven components measured in different units and potentially differing in magnitude by orders. KePIN uses the homoscedastic uncertainty weighting of Kendall et al. [4]: each component k is weighted by e^{-s_k} with a regulariser $\frac{1}{2}s_k$, where $s_k = \log \sigma_k^2$ is a learnable log-variance. The total loss is:

$$\mathcal{L}_{\text{total}} = \sum_k \left(e^{-s_k} \mathcal{L}_k + \tfrac{1}{2} s_k \right) \tag{8}$$

As a component's uncertainty grows, its weight decreases while the regulariser prevents all uncertainties from diverging. This eliminates manual cross-validation of loss weights across domains.

How to read Eq. (8). The scalar s_k is learned jointly with the model parameters. If the optimiser increases s_k, the multiplicative weight e^{-s_k} decreases and thus down-weights $\mathcal{L}_k$; if s_k decreases, $\mathcal{L}_k$ is up-weighted. The additive term $\frac{1}{2}s_k$ discourages trivially driving $s_k \to \infty$ (which would otherwise make $e^{-s_k} \to 0$ and effectively switch off that component). In practice, this mechanism provides an automatic, data-driven balancing of heterogeneous objectives without hand-tuning per-domain coefficients.

Derivation (negative log-likelihood view). This weighting can be motivated by a simple probabilistic model. Suppose each loss component $\mathcal{L}_k$ corresponds (up to constants) to a squared residual under a Gaussian with unknown variance σ_k^2, so that the likelihood contributes a term of the form

$$\mathcal{L}_k(\theta, \sigma_k) \propto \frac{1}{2\sigma_k^2} \mathcal{L}_k(\theta) + \frac{1}{2} \log \sigma_k^2. \tag{9}$$

Letting $s_k := \log \sigma_k^2$ gives $\sigma_k^2 = e^{s_k}$ and therefore $\frac{1}{2\sigma_k^2} = \frac{1}{2} e^{-s_k}$, producing

$$\mathcal{L}_k(\theta, s_k) \propto e^{-s_k} \mathcal{L}_k(\theta) + \tfrac{1}{2} s_k, \tag{10}$$

which is exactly the form used in Eq. (8) (up to an overall constant factor). Intuitively, tasks (or loss components) that are intrinsically noisier are assigned larger uncertainty σ_k^2 and therefore smaller weight.

1.4 Stability Guarantees and Useful Proofs

This section provides short, self-contained proofs for the key stability statements used to justify the Koopman constraints.

Proposition 1 (Contraction under a spectral-norm bound). If $\|\mathbf{K}\|_2 \leq s < 1$, then for any horizon $h \in \mathbb{N}$ and any latent state $\mathbf{z} \in \mathbb{R}^{d_z}$,

$$\|\mathbf{K}^h \mathbf{z}\|_2 \leq s^h \, \|\mathbf{z}\|_2. \tag{11}$$

Proof. By submultiplicativity of the induced matrix norm, $\|\mathbf{K}^h \mathbf{z}\|_2 \leq \|\mathbf{K}^h\|_2 \, \|\mathbf{z}\|_2$ and $\|\mathbf{K}^h\|_2 \leq \|\mathbf{K}\|_2^h \leq s^h$. Combining yields Eq. (11). □

Corollary 1 (Bounded multi-step rollouts and asymptotic decay). Under the same condition $\|\mathbf{K}\|_2 \leq s < 1$, the linear rollout is uniformly bounded and satisfies $\|\mathbf{K}^h \mathbf{z}\|_2 \to 0$ as $h \to \infty$. **Proof.** Immediate from Eq. (11) and the fact that $s^h \to 0$ when $s < 1$. □

Proposition 2 (Spectral radius is bounded by any induced norm). Let $\rho(\mathbf{K})$ denote the spectral radius (maximum eigenvalue magnitude). Then

$$\rho(\mathbf{K}) \leq \|\mathbf{K}\|_2. \tag{12}$$

Proof. Let λ be any eigenvalue with eigenvector $\mathbf{v} \neq 0$, so $\mathbf{K}\mathbf{v} = \lambda \mathbf{v}$. Then $|\lambda| \, \|\mathbf{v}\|_2 = \|\lambda \mathbf{v}\|_2 = \|\mathbf{K}\mathbf{v}\|_2 \leq \|\mathbf{K}\|_2 \, \|\mathbf{v}\|_2$. Dividing by $\|\mathbf{v}\|_2$ gives $|\lambda| \leq \|\mathbf{K}\|_2$ for every eigenvalue; taking the maximum yields Eq. (12). □

Interpretation for KePIN. Proposition 1 explains why enforcing $\|\mathbf{K}\|_2 < 1$ is a strong, conservative stability guarantee: it implies contraction of all rollouts in the Euclidean norm. Proposition 2 explains why spectral-norm control is sufficient to keep eigenvalues inside the unit circle in principle. In practice, because $\mathbf{K}$ is learned with approximate orthonormalisation and finite-sample stochastic optimisation, the eigenvalue-margin penalty (Eq. (3)) provides an additional stabilising gradient signal that complements the hard singular-value constraint.

What we can and cannot "prove" about optimisation. The above results prove properties of the learned linear dynamics *given* a bound on $\|\mathbf{K}\|_2$. They do not constitute a proof that gradient-based training globally converges to the optimal parameters, because the overall objective is non-convex. Empirically, the convergence plots in the supplementary figures indicate that these regularisers are effective at producing stable, consistent operators during training.

2 Training curve Figures

Overview. This section provides supporting qualitative evidence for the optimisation behaviour of KePIN and the effect of the proposed regularisers. Figure 1 summarises end-to-end training/validation convergence, while Fig. 2 decomposes the composite objective into its individual terms to show how the physics-aware constraints behave over training.

Training Convergence (RMSE curves). We track both training and validation RMSE to verify stable optimisation and to visualise the effect of warm restarts and Stochastic Weight Averaging (SWA) on convergence.

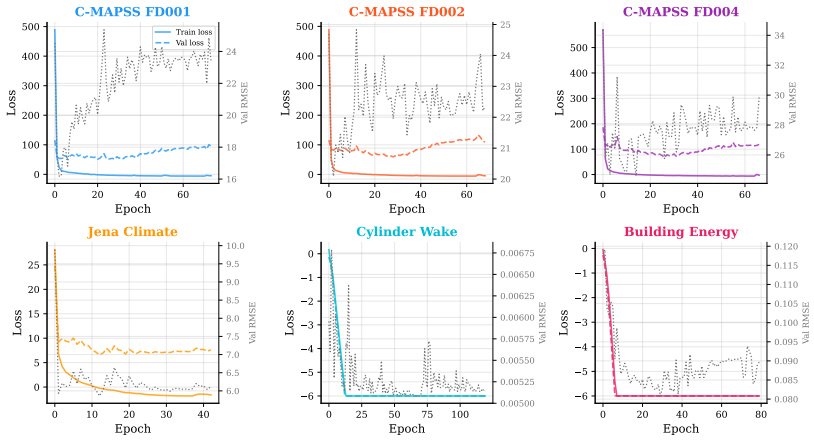

Fig. 1. Training and validation RMSE convergence for KePIN on C-MAPSS FD001 and Jena Climate. Cosine annealing with warm restarts produces periodic learning rate resets that help escape local minima. The Stochastic Weight Averaging (SWA) phase (shaded) stabilises the final model checkpoint.

Loss-Component Dynamics (constraint activation). We plot each loss term separately to show how Koopman consistency, multi-step rollout, and spectral stability evolve during optimisation, and to confirm that the stability-related penalty becomes inactive once the learned operator is well-behaved.

3 Baselines

To ensure fair and reproducible comparison, we re-implemented six baseline architectures under identical preprocessing, feature selection, normalisation, and evaluation protocols:

- **MLP**: Three-layer fully connected network (256–128–64 units, ReLU, dropout 0.3).
- **LSTM** [8]: Single-layer LSTM (128 units) with dense prediction head.
- **BiLSTM** [7]: Bidirectional LSTM (128 units per direction) with dense head.
- **CNN-LSTM**: Stacked Conv1D feature extractor followed by an LSTM decoder.
- **Transformer** [6]: Multi-head self-attention encoder (4 heads) with positional encoding and dense head.
- **Vanilla FCN**: Conv1D blocks with Batch Normalisation and SE attention, without the Koopman module or physics-informed losses—equivalent to KePIN's encoder used as a standalone predictor.

All baselines use the same Adam optimiser, learning rate schedule, batch size, and early stopping criteria to isolate the contribution of KePIN's Koopman module and composite loss.

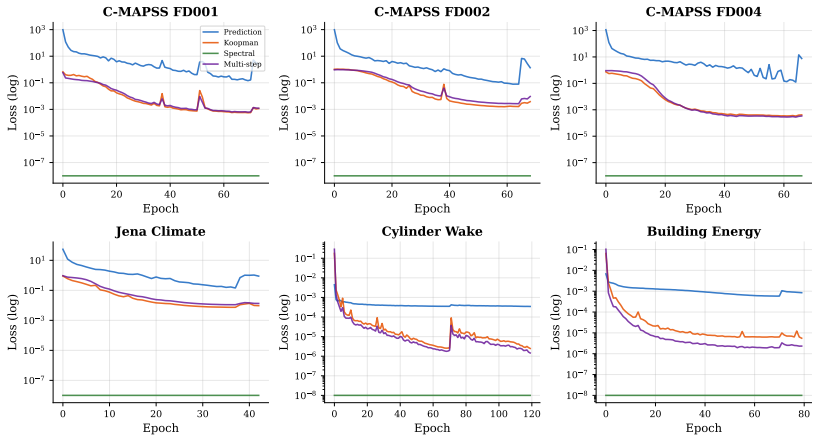

Fig. 2. Evolution of each individual loss component during training on C-MAPSS FD001. The Koopman consistency loss $\mathcal{L}_{\text{koop}}$ and multi-step rollout loss $\mathcal{L}_{\text{multi}}$ decrease monotonically, indicating that the encoder successfully discovers a Koopman-compatible latent space. The spectral stability loss $\mathcal{L}_{\text{spec}}$ rapidly converges to near zero, confirming that eigenvalues remain within the unit circle throughout training.

4 Training Pipeline

This section describes the end-to-end experimental pipeline used to train KePIN and the re-implemented baselines in a consistent manner across domains. The goal is to ensure that performance differences are attributable to modelling choices (Koopman module and physics-aware losses) rather than to mismatched preprocessing or evaluation.

4.1 Exact Pipeline (Formulations)

Sliding-window construction. Given a multivariate time series $\mathbf{x}_{1:L}$ with $\mathbf{x}_t \in \mathbb{R}^d$, we form overlapping windows of length T:

$$\mathbf{X}_t := [\mathbf{x}_t, \mathbf{x}_{t+1}, \ldots, \mathbf{x}_{t+T-1}] \in \mathbb{R}^{T\times d}, \qquad t = 1, \ldots, L - T + 1. \tag{13}$$

The supervised label for the window is the target at the last index (as in the loader): $y_t := y_{t+T-1}$.

C-MAPSS target construction (RUL). For each engine/unit u with maximum observed cycle $c_u^{\max}$, the raw RUL target at cycle c is

$$y_u(c) = c_u^{\max} - c. \tag{14}$$

We apply the standard cap (used throughout our C-MAPSS experiments):

$$y_u^{\text{cap}}(c) = \min\{y_u(c),\ 125\}. \tag{15}$$

Feature normalisation (fit on train only). For each feature dimension j, we compute mean μ_j and standard deviation s_j on the training split only, and standardise

$$x'_{t,j} = \frac{x_{t,j} - \mu_j}{s_j + \varepsilon}, \qquad \varepsilon \approx 10^{-8}. \tag{16}$$

EMA smoothing. Before windowing/training, we apply an exponential moving average (EMA) along the time axis:

$$\tilde{\mathbf{x}}_t = \alpha\, \mathbf{x}_t + (1-\alpha)\, \tilde{\mathbf{x}}_{t-1}, \qquad \tilde{\mathbf{x}}_1 = \mathbf{x}_1. \tag{17}$$

In our training code, if α is not specified, it is auto-tuned from the window length T as

$$\alpha = \mathrm{clip}\Big(\frac{2}{1+T},\ 0.05,\ 0.5\Big). \tag{18}$$

Learning-rate schedule (warmup + cosine warm restarts). For the optimised C-MAPSS training, we use linear warmup for E_w epochs followed by cosine annealing with warm restarts. Let η_0 be the initial/peak learning rate and $\eta_{\min}$ the minimum learning rate. Then

$$\eta(e) = \begin{cases} \eta_0\, \frac{e+1}{E_w}, & 0 \le e < E_w, \\ \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\Big(1 + \cos\big(\pi\, T_{\mathrm{cur}}/T_0\big)\Big), & e \ge E_w, \end{cases} \tag{19}$$

where $T_0 = \max\{\lfloor (E - E_w)/3 \rfloor,\ 40\}$ and $T_{\mathrm{cur}} = (e - E_w) \bmod T_0$.

Gradient clipping. We clip gradients by the global norm with threshold $c = 2.0$:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|_2}\right). \tag{20}$$

Gaussian feature noise augmentation. For C-MAPSS, we inject additive noise during training:

$$\mathbf{x}'_t = \mathbf{x}_t + \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0},\ \sigma^2_{\mathrm{noise}}\mathbf{I}). \tag{21}$$

Mixup (time-series). Given two windows $(\mathbf{X}_a, y_a)$ and $(\mathbf{X}_b, y_b)$ in the same mini-batch, we form the mixed sample

$$\mathbf{X}_{\mathrm{mix}} = \lambda\, \mathbf{X}_a + (1-\lambda)\, \mathbf{X}_b, \qquad y_{\mathrm{mix}} = \lambda\, y_a + (1-\lambda)\, y_b. \tag{22}$$

In our implementation, $\lambda \sim \mathrm{Unif}(0,1)$ and we apply $\lambda \leftarrow \max\{\lambda, 1-\lambda\}$ to bias mixes towards one of the original samples; mixup starts only after the warmup phase.

Label smoothing near the RUL cap. To soften the hard kink near the RUL cap, we perturb only near-cap labels. Let $y_{\max}$ be the maximum label value in the training set (for capped C-MAPSS, $y_{\max} = 125$). For labels satisfying $y \ge 0.9\, y_{\max}$, we apply

$$y' = \mathrm{clip}(y + \eta,\ 0,\ y_{\max}), \qquad \eta \sim \mathcal{N}(0,\ (0.5\, \sigma_{\mathrm{label}})^2). \tag{23}$$

Stochastic Weight Averaging (SWA) and model selection. Starting at epoch $e_{\text{SWA}} = \lfloor \gamma E \rfloor$ (with fraction $\gamma \in (0, 1)$), we maintain a running average of weights:

$$\theta_{\text{SWA}}^{(k)} = \frac{(k-1)\,\theta_{\text{SWA}}^{(k-1)} + \theta^{(k)}}{k}. \tag{24}$$

At the end of training, we evaluate both the best-validation checkpoint and the SWA weights and keep whichever achieves lower validation RMSE.

Ensembling. When reporting ensemble results (C-MAPSS), we train $n_{\text{runs}} = 3$ independent runs with different random seeds and average predictions:

$$\hat{y}(\mathbf{X}) = \frac{1}{n_{\text{runs}}} \sum_{r=1}^{n_{\text{runs}}} \hat{y}^{(r)}(\mathbf{X}). \tag{25}$$

4.2 Exact Hyperparameters (as used for the reported C-MAPSS results)

Per-subset training settings. For C-MAPSS FD001–FD004, our optimised training uses the following fixed values (from the experiment configuration used to generate the paper results):

- **FD001**: $E = 300$, $P = 50$, batch size = 128, $\eta_0 = 8 \times 10^{-4}$, $\eta_{\min} = 10^{-6}$, $E_w = 8$, SWA fraction $\gamma = 0.75$, mixup $\alpha = 0.2$, $\sigma_{\text{noise}} = 0.01$, $\sigma_{\text{label}} = 2.0$, $n_{\text{runs}} = 3$.
- **FD002**: $E = 250$, $P = 45$, batch size = 256, $\eta_0 = 6 \times 10^{-4}$, $\eta_{\min} = 10^{-6}$, $E_w = 5$, SWA fraction $\gamma = 0.80$, mixup $\alpha = 0.15$, $\sigma_{\text{noise}} = 0.008$, $\sigma_{\text{label}} = 1.5$, $n_{\text{runs}} = 3$.
- **FD003**: $E = 300$, $P = 50$, batch size = 128, $\eta_0 = 8 \times 10^{-4}$, $\eta_{\min} = 10^{-6}$, $E_w = 8$, SWA fraction $\gamma = 0.75$, mixup $\alpha = 0.2$, $\sigma_{\text{noise}} = 0.012$, $\sigma_{\text{label}} = 2.0$, $n_{\text{runs}} = 3$.
- **FD004**: $E = 250$, $P = 45$, batch size = 256, $\eta_0 = 6 \times 10^{-4}$, $\eta_{\min} = 10^{-6}$, $E_w = 5$, SWA fraction $\gamma = 0.80$, mixup $\alpha = 0.15$, $\sigma_{\text{noise}} = 0.008$, $\sigma_{\text{label}} = 1.5$, $n_{\text{runs}} = 3$.

Here E denotes the maximum epochs and P the early-stopping patience (in epochs). The schedule and augmentations are defined by Eqs. (19)–(25).

Multi-domain (unified/ablation) defaults. For the multi-domain ablation runs (C-MAPSS + Jena Climate) executed with the unified runner, we use the following exact defaults:

- **Degradation domains (default)**: $E = 200$, $P = 40$, batch size = 128, $\eta_0 = 8 \times 10^{-4}$.
- **Weather (Jena Climate)**: $E = 250$, $P = 50$, batch size = 256, $\eta_0 = 5 \times 10^{-4}$.
- **Overrides for FD002/FD004 (ablation runner)**: batch size = 256, $\eta_0 = 6 \times 10^{-4}$.

These runs use the same preprocessing/EMA (Eqs. (16)–(18)) and the same composite loss, but do not necessarily enable the full C-MAPSS-specific warmup/mixup/SWA recipe above.

4.3 Data Preprocessing

Cleaning and feature handling. For each dataset we (i) construct fixed-length input windows of length T (Table in the main paper), (ii) standardise input features using statistics computed on the training split only, and (iii) apply the same feature selection and normalisation across all methods.

Smoothing. To reduce high-frequency sensor noise, we apply exponential moving average (EMA) smoothing to the raw time-series inputs before windowing. This is applied identically for KePIN and all baselines.

4.4 Splits and Windowing

Train/validation/test protocol. We construct training, validation, and test sets following the dataset's standard protocol (e.g., engine-wise splits for C-MAPSS). Early stopping and model selection use only the validation split; the test split is used once for final reporting.

Window construction. Given a multivariate sequence $\mathbf{x}_{1:L}$, we extract windows $\mathbf{x}_{t:t+T-1}$ with a fixed stride (domain-dependent) and pair each window with its corresponding target value at the prediction horizon. This produces an i.i.d.-style training set while preserving temporal structure within each window.

4.5 Model Configuration

Automatic capacity selection. KePIN selects a model tier (small/medium/large) based on input dimensionality d. This avoids per-dataset manual tuning of architecture size and keeps the training recipe consistent across domains.

Domain-aware loss activation. The training code automatically activates either degradation mode (7 loss components) for predictive maintenance tasks or forecasting mode (4 components) for all other datasets. This preserves a single implementation across domains while enforcing appropriate physics/operational constraints.

4.6 Optimisation and Model Selection

Optimiser and schedule. We train with Adam and use linear warmup followed by cosine annealing with warm restarts. Gradients are globally clipped to stabilise optimisation.

Early stopping. We monitor validation RMSE and stop training when it does not improve for a fixed patience window. This prevents overfitting and standardises stopping across methods.

Stochastic Weight Averaging (SWA). During the final portion of training, we maintain an SWA weight average and select the better of the best validation checkpoint and the SWA model. SWA typically improves generalisation by smoothing sharp minima.

Ensembling and final metrics. For settings where we report ensembles, we train multiple independent runs with different random seeds and average their predictions. We report RMSE, MAE, and R^2 on the test set.

5 Implementation Details

KePIN is implemented in TensorFlow/Keras 2.x [1]. Training uses the Adam optimiser [3] with initial learning rate 8×10^{-4}, linear warmup over 8 epochs, cosine annealing with warm restarts [5], global gradient clipping (norm 2.0), and early stopping with patience 40–50 epochs on validation RMSE. Stochastic Weight Averaging (SWA) is applied over the final 25% of training epochs to improve generalisation. For C-MAPSS, Gaussian noise augmentation ($\sigma = 0.01$), mixup interpolation ($\alpha = 0.2$), label smoothing near the RUL cap, and 3-run prediction ensembling are applied. All time-series inputs undergo exponential moving average (EMA) smoothing before training. Experiments run on an NVIDIA A100-PCIE-40GB GPU. Results are reported as RMSE, MAE, and R^2 averaged over n_{runs} independent runs when ensembling is enabled (for C-MAPSS, $n_{\text{runs}} = 3$), unless stated otherwise.

References

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., et al.: Tensorflow: Large-scale machine learning on heterogeneous distributed systems. arXiv preprint arXiv:1603.04467 (2016)
2. Brunton, S.L., Budišić, M., Kaiser, E., Kutz, J.N.: Modern koopman theory for dynamical systems. arXiv preprint arXiv:2102.12086 (2021)
3. Diederik, K.: Adam: A method for stochastic optimization. (No Title) (2014)
4. Kendall, A., Gal, Y., Cipolla, R.: Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 7482–7491 (2018)
5. Loshchilov, I., Hutter, F.: Sgdr: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983 (2016)
6. Mo, Y., Wu, Q., Li, X., Huang, B.: Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. Journal of Intelligent Manufacturing **32**(7), 1997–2006 (October 2021). https://doi.org/10.1007/s10845-021-01750-x, https://ideas.repec.org/a/spr/joinma/v32y2021i7d10.1007_s10845-021-01750-x.html

7. Wang, J., Wen, G., Yang, S., Liu, Y.: Remaining useful life estimation in prognostics using deep bidirectional lstm neural network. In: 2018 Prognostics and System Health Management Conference (PHM-Chongqing). pp. 1037–1042 (2018). https://doi.org/10.1109/PHM-Chongqing.2018.00184
8. Zheng, S., Ristovski, K., Farahat, A.K., Gupta, C.: Long short-term memory network for remaining useful life estimation. 2017 IEEE International Conference on Prognostics and Health Management (ICPHM) pp. 88–95 (2017), https://api.semanticscholar.org/CorpusID:6017005